Making `std::underlying_type` SFINAE-friendly

Document number:	P0340R1
Date:	2018-05-07
Audience:	LEWG → LWG

Reply to: R. "Tim" Song <rs2740@gmail.com>

Revision history

- R1 (2018-05-07): Updated wording to current WP and added alternative wording options.

Abstract

This paper proposes making `std::underlying_type` SFINAE-friendly. In particular, it would make instantiating `std::underlying_type<T>` for a non-enumeration type `T` well-defined and result in an empty struct.

Motivation

Currently, `std::underlying_type<T>` requires `T` to be a complete enumeration type. instantiating it with any other type results in undefined behavior, typically a hard error. This makes it tricky to use in SFINAE contexts. For example, if one wants to write a function template that want to constrain a template argument to "enumeration type whose underlying type is `int`", the "obvious" approach would be something along the lines of

```
template<class T>
std::enable_if_t<std::is_enum_v<T> &&
                 std::is_same_v<std::underlying_type_t<T>, int>> foo(T t);
```

Unfortunately, this won't work; writing `foo(0)` will almost certainly result in a hard error, even if there is a `void foo(int);` overload available. Instead, actual evaluation of `std::underlying_type<T>` must be deferred until `T` is known to be an enum, with something like

```
template<class T>
std::enable_if_t<std::is_same_v<typename std::enable_if_t<std::is_enum_v<T>, std::underlying_type<T>>::type, int>> foo(T t);
```

Not only is this harder to write (`typename ...::type`), it is also harder to understand (nested `enable_if`s).

This is a recurring problem on StackOverflow; see for example 1, 2, 3, all from 2016.

If `std::underlying_type<T>` is well-defined but has no member `type` when `T` is not an enumeration type, then the above function template can be simplified to

```
template<class T>
std::enable_if_t<std::is_same_v<std::underlying_type_t<T>, int>> foo(T t);
```

which is shorter, more intuitive, and easier to understand.

It's worth noting that libc++ has a `__sfinae_underlying_type` for internal use with an implementation very similar to that described below (but with an additional helper member typedef).

Impact on the standard

This is a pure extension, as currently instantiating `std::underlying_type` over a non-enumeration type results in undefined behavior, typically a compile-time error.

Design alternatives

A possible alternative would be to make the nested typedef `type` return the type unchanged if `T` is not an enumeration type, matching the behavior of some other TransformationTraits such as `add_lvalue_reference` and `remove_extent`. This approach also avoids hard errors. This author, however, does not see reasons to make `underlying_type_t<int>` well-formed.

Another design alternative would be to support `char16_t`, `char32_t`, and `wchar_t`, each of which also has a "underlying type" (see [basic.fundamental]/5). And though the standard doesn't use the term, one might say that plain `char` similarly has an "underlying

type" of either `signed char` or `unsigned char` depending on the implementation. The current specification in the standard permits implementations to support those types as an extension, though the author is not aware of any implementation that does so. In the author's opinion, the two categories are sufficiently distinct that lumping them into the same type trait would not be advisable.

The prohibition against incomplete enumeration types is left undisturbed, given the potential for ODR violations, and because the benefit from supporting such types is minimal at best.

Proposed wording

To facilitate discussion, wording is provided for all three possible variations of supported types (enumeration only, types with a core language "underlying type" only, and types with a core language "underlying type" plus `char`) and both possible handling of unsupported types (no member `type` and pass-through).

◉Support enumeration types only
◯Support wide character types (`wchar_t`, `char16_t`, `char32_t`) and enumeration types
◯Support `char`, wide character types, and enumeration types
☐Pass through other types (rather than having no `type`)

Edit the table in [meta.trans.other] as indicated:

Template	Comments
...	...
`template <class T> struct underlying_type;`	**If `T` is an enumeration type, the** ~~The~~ member typedef `type` shall name the underlying type of `T` **(10.2 [dcl.enum]); otherwise, there shall be no member `type`**. *Requires:* ~~`T` shall be a complete enumeration type (10.2)~~ **If `T` is an enumeration type, `T` shall be a complete type.**
...	...

Possible implementation

Given an intrinsic `__underlying_type(T)` (which is already needed to implement the current version of `underlying_type`), the implementation of the enumeration-only version is trivial:

```
template<class T, bool = std::is_enum_v<T>> struct _Underlying_type {};
template<class T> struct _Underlying_type<T, true> { using type = __underlying_type(T); };

template<class T> struct underlying_type : _Underlying_type<T> { };
```

Supporting character types is simply a matter of adding a few specializations, and supporting pass-through behavior simply requires adding a `using type = T;` to the primary `_Underlying_type` template.